

Merged

RegexHighlighter.collectMatches und RegexHighlighter.resolveConflicts implementieren und alle geforderten Testfälle erstellt #2[zouse187](#) merged 3 commits into [master](#) from [feature/regexhighlighter](#)

Code ▾

RegexHighlighter.collectMatches und RegexHighlighter.resolveConflicts implementieren und alle geforderten Testfälle erstellt #2

Merged

[zouse187](#) merged 3 commits into [master](#) from [feature/regexhighlighter](#) yesterday

Conversation 1



Commits 3



Checks 3



Files changed 4

**zouse187** commented [3 days ago](#)

Owner

Dieser PR implementiert Aufgabe 2: RegexHighlighter.

Enthalten:

- collectMatches: Wendet alle Tokens aus MiniJavaTokens auf den Text an
- resolveConflicts: Entfernt überlappende Regionen gemäß Aufgabenblatt
- vollständige JUnit-Tests für einfache, überlappende und angrenzende Regionen

Status:

- Implementierung fertig
- Tests grün

**zouse187** added 2 commits [3 days ago](#)[RegexHighlighter.collectMatches und RegexHighlighter.resolveConflicts...](#) ...[723449c](#)[Code mit spotless formatiert](#)✓ [20afbc0](#)**Ph4ntomic** commented [3 days ago](#) • edited ▾

Ich habe mir den Pull Request angeschaut. Die Lösung wirkt insgesamt sauber und passend zur Aufgabe.

Die Implementierung von `collectMatches` ist gut nachvollziehbar: Die Treffer der einzelnen Regex-Regeln werden gesammelt und als Highlight-Regionen zurückgegeben.

Auch `resolveConflicts` ist verständlich umgesetzt und entfernt überlappende Bereiche so, dass die vorher priorisierten Regionen erhalten bleiben.

Merged

RegexHighlighter.collectMatches und RegexHighlighter.resolveConflicts implementieren und alle geforderten Testfälle erstellt #2[zouse187](#) merged 3 commits into [master](#) from [feature/regexhighlighter](#)

Einen kleinen Punkt würde ich vor dem Merge noch korrigieren: In `RegexHighlighterTest` fehlt bei `regionsNextToEachother_doNotOverlap()` vermutlich die Annotation `@Test`.

Ohne diese Annotation wird die Methode von JUnit nicht als Test ausgeführt.

Inhaltlich ist der Test aber wichtig, weil er prüft, dass direkt angrenzende Highlight-Regionen nicht fälschlich als Überlappung behandelt werden.

Eine kleine fachliche Ergänzung: `resolveConflicts` vergleicht jede neue Region mit den bereits übernommenen Regionen.

Dadurch hat die Methode im ungünstigen Fall eine Laufzeit von $O(n^2)$.

Für diese Aufgabe und typische Eingaben ist das völlig in Ordnung, aber es ist gut, das im Hinterkopf zu behalten.

Wenn die `@Test`-Annotation ergänzt ist und `./gradlew test` sowie `./gradlew build` sauber durchlaufen, wirkt der Pull Request aus meiner Sicht solide und gut mergebar.

[@zouse187](#)[@test](#) [Notation an Test-Methode hinzugefügt](#)✓ [87763f5](#)[zouse187](#) merged commit [c43f914](#) into [master](#) [yesterday](#)
3 checks passed[View details](#)**Pull request successfully merged and closed**

You're all set — the branch has been merged.

Reviewers

No reviews

Assignees

No one assigned

Labels

None yet

Projects

None yet

Merged**RegexHighlighter.collectMatches und RegexHighlighter.resolveConflicts implementieren und alle geforderten Testfälle erstellt #2**[zouse187](#) merged 3 commits into [master](#) from [feature/regexhighlighter](#)**Development**

Successfully merging this pull request may close these issues.

None yet

2 participants